

Strings as Sequences

Lecture 19

CS 1001

Kirk Duran

Today

- ▶ Model a string as an ordered sequence of characters.
- ▶ Create strings using quotation marks and escape sequences.
- ▶ Access characters using positive and negative indexes.
- ▶ Measure and slice strings using the same positional model used for lists.
- ▶ Explain string immutability and distinguish mutation from rebinding.
- ▶ Combine, compare, search, split, join, and transform strings.
- ▶ Connect Python strings to Unicode code points.

Key idea

A string is an immutable sequence: its character positions can be read and sliced, but an existing string value cannot be changed in place.

Part 1: Strings Are Sequential Data

Definition: String

Definition

A **string** is a Python sequence value representing text. A string contains an ordered sequence of Unicode characters.

Example

```
text = "Python"
```

- ▶ The sequence has a fixed order.
- ▶ Each position contains one character.
- ▶ Character positions are identified by integer indexes.
- ▶ The empty string contains zero characters: "".
- ▶ Text such as "42" is still a string; Python has no separate built-in character type.

The Same Sequence Model Applies to Lists and Strings

List

[10, 20, 30, 40]

- ▶ ordered positions
- ▶ integer indexes
- ▶ slicing
- ▶ mutable

String

"code"

- ▶ ordered positions
- ▶ integer indexes
- ▶ slicing
- ▶ immutable

Key idea

Lists and strings share positional sequence operations, but differ fundamentally in whether the existing value may be changed.

A Useful Mental Model: Adjacent Character Positions

String

"Python"

index	0	1	2	3	4	5
char.	P	y	t	h	o	n

Image Placeholder

string-adjacent-positions.png

Key idea

For tracing, think of a string as consecutive character positions numbered from left to right.

Creating String Literals

Single-line strings

```
name = "Ada"  
language = 'Python'
```

Multi-line literal

```
message = """Line one  
Line two"""
```

- ▶ Single and double quotation marks both delimit ordinary string literals.
- ▶ Matching quotation marks must close the literal.
- ▶ Triple quotation marks may span physical source-code lines.

Part 2: Indexing and Length

String Indexing Uses Zero-Based Positions

Example

```
text = "Python"
```

```
text[0] → "P"
```

```
text[1] → "y"
```

```
text[5] → "n"
```

Key idea

Indexing a string produces a new one-character string containing the character at that position.

Negative Indexes Count from the End

Example

```
text = "Python"
```

character	P	y	t	h	o	n
positive	0	1	2	3	4	5
negative	-6	-5	-4	-3	-2	-1

Examples

```
text[-1] → "n"    text[-2] → "o"
```

An Invalid String Index Raises `IndexError`

Example

```
text = "cat"
```

- ▶ Valid positive indexes are 0, 1, and 2.
- ▶ Valid negative indexes are -1, -2, and -3.
- ▶ `text[3]` raises `IndexError`.
- ▶ `text[-4]` also raises `IndexError`.

Key idea

Indexing requires one existing position. The same boundary rule applies to lists and strings.

len() Counts Characters

Definition

`len(string)` returns the number of character positions in the string.

Examples

`len("Python")` → 6

`len("")` → 0

`len("Hello, world!")` → 13

- ▶ Spaces and punctuation count as characters.
- ▶ Length is not the final positive index.

Length and the Final Index

Example

```
text = "Python"
```

```
len(text) → 6
```

```
text[len(text) - 1] → "n"
```

Key idea

For a nonempty string, the final positive index is $\text{len}(\text{text}) - 1$.

Part 3: String Slicing

String Slicing Uses the Same `start:stop:step` Model

General syntax

```
text[start:stop:step]
```

- ▶ `start` is included.
- ▶ `stop` is excluded.
- ▶ `step` controls the distance between selected positions.
- ▶ The result is a new string.

The Stop Boundary Is Still Exclusive

Example

```
text = "Python Programming"
```

```
text[0:6] → "Python"
```

- ▶ Index 0 is selected.
- ▶ Indexes 1 through 5 are selected.
- ▶ Index 6 is the stopping boundary and is not selected.

Key idea

Read 0:6 as “start at 0, stop before 6.”

Omitting Slice Bounds Uses Sequence Boundaries

Examples

```
text = "Python Programming"
```

```
text[:6] → "Python"
```

```
text[7:] → "Programming"
```

```
text[:] → "Python Programming"
```

- ▶ The original string remains unchanged.
- ▶ A slice expression evaluates to another string value.

The Optional Step Changes Which Positions Are Selected

Example

```
text = "abcdefgh"
```

```
text[0:8:2] → "aceg"
```

```
text[1:8:2] → "bdfh"
```

- ▶ The default step is 1.
- ▶ A step of 2 selects every second position within the slice bounds.
- ▶ A step of 0 is invalid and raises `ValueError`.

A Negative Step Produces Characters in Reverse Position Order

Example

```
text = "Python"
```

```
text[::-1] → "nohtyP"
```

- ▶ Omitted bounds adapt to the negative direction.
- ▶ The result is a new reversed string.
- ▶ The original text remains "Python".

Image Placeholder

string-reverse-slice.png

Practice: Predict the Slice Results

Given

```
text = "abcdefghij"
```

In class

Predict each result before evaluating it.

1. `text[2:6]`
2. `text[:4]`
3. `text[6:]`
4. `text[1:9:3]`
5. `text[-4:-1]`
6. `text[::-1]`

Part 4: Immutability and Building New Strings

Strings Are Immutable

Definition

An **immutable** value cannot be changed after that value has been created.

Invalid operation

```
text = "cat"  
text[0] = "b"
```

Result

Python raises `TypeError`: individual string positions cannot be assigned new characters.

Concatenation Creates a New String

Example

```
first = "Hello"  
second = " world"  
message = first + second
```

Values afterward

```
first → "Hello"  
second → " world"  
message → "Hello world"
```

Key idea

The + operator combines string values into a new string; it does not modify either operand.

Concatenation Requires String Operands

Invalid

```
"Age: " + 21
```

Explicit conversion

```
"Age: " + str(21) → "Age: 21"
```

- ▶ `str(value)` produces a string representation of a value.
- ▶ Converting to a string changes the type of the produced value, not the original variable.

Reassignment Is Not Mutation

Code

```
text = "Python"  
text = "C" + text[1:]
```

- ▶ `text[1:]` creates the new string "ython".
- ▶ Concatenation creates the new string "Cython".
- ▶ Assignment then rebinds the name `text` to that new value.
- ▶ The original string value "Python" was never altered.

Key idea

A variable can be rebound even when the value it previously referred to is immutable.

Escape Sequences Represent Special Characters Inside a Literal

Common examples

```
"line one\nline two"  newline  
"name\tvalue"    tab  
"She said \"hello\"."  embedded quotation marks  
"C:\\Users\\Ada"   literal backslashes
```

- ▶ An escape sequence is written using a backslash in source code.
- ▶ The resulting string contains the represented character, not necessarily the literal source characters used to write the escape.

Part 5: Membership, Equality, and Character Codes



Membership Checks Whether a Substring Occurs

Examples

```
text = "Python programming"
```

```
"Python" in text → True
```

```
"gram" in text → True
```

```
"python" in text → False
```

```
"Java" not in text → True
```

- ▶ Membership can search for one character or an entire substring.
- ▶ The comparison is case-sensitive.
- ▶ Both `in` and `not in` produce Boolean values.

String Equality Requires the Same Characters in the Same Order

Examples

`"Hello" == "Hello" → True`

`"Hello" == "hello" → False`

`"cat" == "cat " → False`

`"abc" != "acb" → True`

Key idea

Equality compares string contents exactly; case, order, spaces, and punctuation matter.

Ordering Comparisons Are Lexicographic

Definition

String ordering compares corresponding characters from left to right until a difference determines the result; if one string is a prefix of the other, the shorter string orders first.

Examples

`"apple" < "banana" → True`

`"app" < "apple" → True`

- ▶ Character ordering is based on Unicode code points.
- ▶ Therefore, ordering is case-sensitive and may not match dictionary conventions used by humans.

Python Strings Use Unicode

- ▶ Python's `str` type represents Unicode text.
- ▶ Unicode assigns each character a numerical **code point**.
- ▶ ASCII characters occupy the first 128 Unicode code points.

Examples

```
ord("A") → 65  
ord("Z") → 90  
ord("a") → 97  
chr(65) → "A"
```

Key idea

Because uppercase and lowercase letters have different code points, relational comparisons are case-sensitive; for example, `"Zebra" < "apple"` is `True`.

Part 6: Converting and Transforming Strings

`list(string)` Produces a List of One-Character Strings

Example

```
text = "hello"  
chars = list(text)
```

```
chars → ["h", "e", "l", "l", "o"]
```

- ▶ The original string remains unchanged.
- ▶ The result is a new list.
- ▶ Because lists are mutable, the resulting character positions can later be changed as list elements.

split() Produces a List of Substrings

Default whitespace separation

```
text = "hello world Python"  
text.split() → ["hello", "world", "Python"]
```

Explicit separator

```
"red,green,blue".split(",")  
→ ["red", "green", "blue"]
```

Key idea

split() does not modify the source string; it returns a new list containing string pieces.

`join()` Combines a List of Strings

Example

```
words = ["hello", "world", "Python"]  
sentence = " ".join(words)  
  
sentence → "hello world Python"
```

- ▶ The string before `.join(...)` is the separator.
- ▶ The list supplied to `join()` must contain strings.
- ▶ The result is a new string.

Case-Conversion Methods Return New Strings

Examples

```
text = "hello world"
```

```
text.upper() → "HELLO WORLD"
```

```
text.lower() → "hello world"
```

```
text.capitalize() → "Hello world"
```

```
text.title() → "Hello World"
```

Important

None of these methods mutates text; each produces a new string value.

Formatted String Literals Build Strings from Expressions

Example

```
name = "Ada"  
age = 36  
message = f"Name: {name}, Age: {age}"
```

Result

```
message → "Name: Ada, Age: 36"
```

- ▶ Expressions inside braces are evaluated first.
- ▶ Their formatted representations are inserted into the new string.
- ▶ Format specifications may control representation, e.g. `f"{3.14159:.2f}"` produces `"3.14"`.
- ▶ Formatting changes the produced text, not the underlying numeric value.

Integrated Trace: New Strings, No Mutation

Code

```
text = "python"  
head = text[:2]  
tail = text[2:]  
label = head.upper() + tail  
words = label.split("h")  
joined = "-".join(words)
```

In class

Determine the final values of `text`, `head`, `tail`, `label`, `words`, and `joined`. Identify every expression that creates a new value.

Strings in Memory: Conceptual Model vs. Implementation

- ▶ Our tracing model treats a string as adjacent character positions.
- ▶ Python guarantees the sequence behavior: indexing, slicing, length, equality, and immutability.
- ▶ The exact physical representation of Unicode strings is an implementation detail.
- ▶ An implementation may use compact internal encodings rather than one fixed-size memory cell per visible character.

Image Placeholder

`string-conceptual-vs-implementation.png`

Operation Summary

Operation	Result
<code>text[i]</code>	one-character string
<code>text[start:stop:step]</code>	new string
<code>len(text)</code>	integer length
<code>a + b</code>	new concatenated string
<code>sub in text</code>	Boolean
<code>text.split(...)</code>	new list of strings
<code>sep.join(parts)</code>	new string
<code>text.upper() / lower()</code>	new string

Takeaways

- ▶ A string is an ordered, indexed, immutable sequence of Unicode characters.
- ▶ String indexing and slicing use the same positional rules as lists, including negative indexes and an exclusive stop boundary.
- ▶ Concatenation, slicing, case conversion, formatting, and related operations produce new string values.
- ▶ Reassigning a variable to a new string is not mutation of the old string.
- ▶ Membership, equality, and relational comparisons are case-sensitive.
- ▶ `split()` converts text into a list of string pieces; `join()` combines string pieces into a new string.

Key idea

When tracing string code, track values and bindings carefully: the string value itself never changes in place.